# Database Guide

AI Content Creator - backend schema for new developers

PostgreSQL 16 | SQLAlchemy 2.0 models in `backend/app/models/` | schema owned by Alembic (`backend/alembic/versions/`)

13 tables. Every primary key is a UUID v4. Every timestamp is TIMESTAMPTZ (UTC).

# 1. The big picture

The product is a B2B social-media tool: a team logs in, describes the brands it posts for, writes posts, gets them approved, and publishes them to connected Facebook / Instagram / LinkedIn accounts. The schema follows that story top to bottom:

| Layer | Tables | Question it answers |
|---|---|---|
| Identity | users, refresh_tokens | Who is logged in? |
| Tenant | workspaces, workspace_members | Which team, and what may each person do? |
| Brand profile | brands | For whom is this post written? (tone, audience, logo) |
| Content | posts, post_media, approval_logs | What is being published, and who signed it off? |
| Channels | social_accounts, publish_jobs | Where does it go, and did it get there? |
| Operational | usage_logs, caption_templates, audit_events | Telemetry, seed content, compliance ledger |

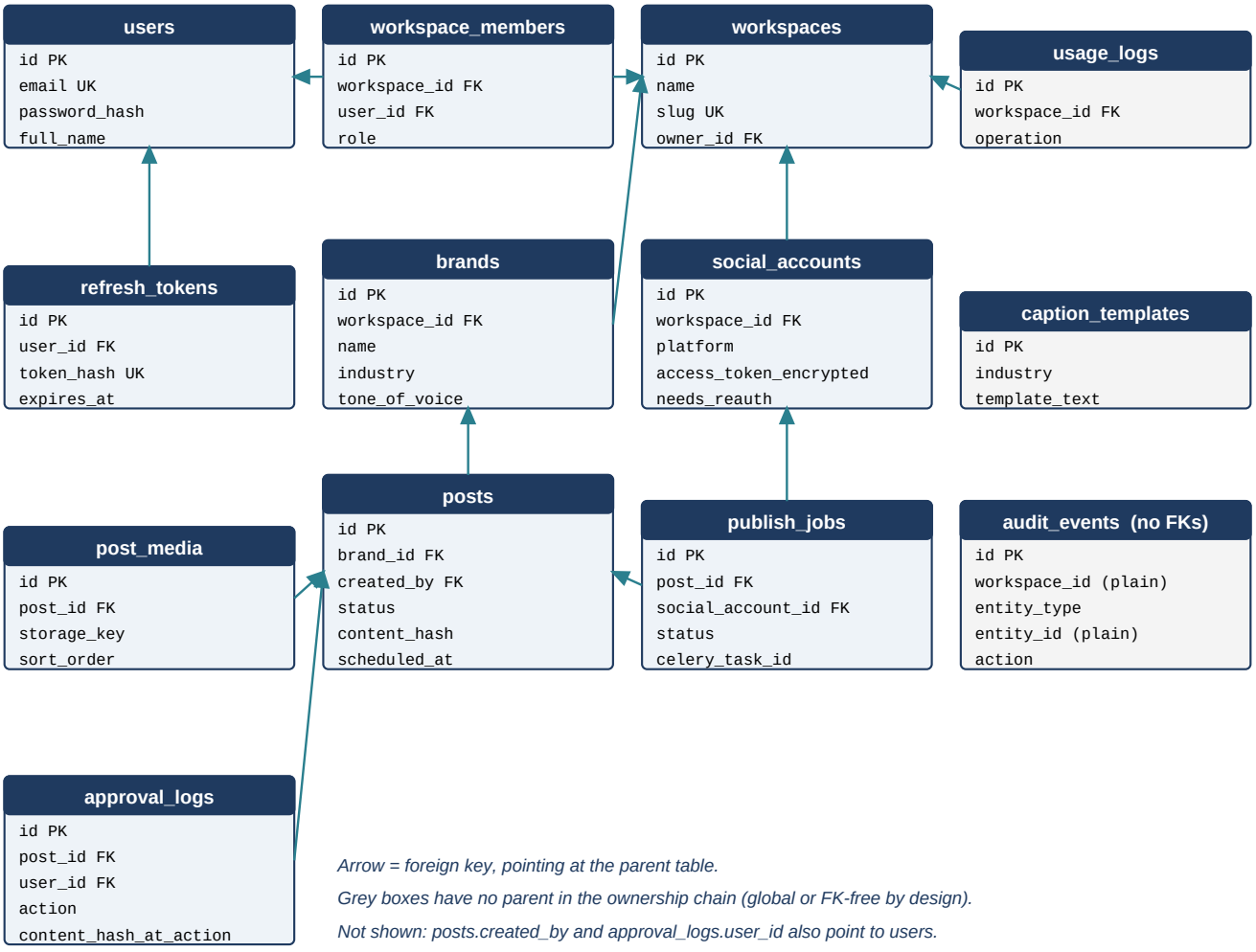
### *Ownership chain*

`workspace -> brand -> post -> post_media / approval_logs / publish_jobs`. Everything a customer owns can be reached from their workspace. Deleting a workspace cascades down this chain (except `audit_events`, which deliberately survives).

**Tenancy rule for every query:** a user may only touch a row if they are in `workspace_members` for the workspace that row rolls up to. The API enforces this in one place, `app/core/dependencies.py`, never in individual routes.

# 2. Entity-relationship diagram

Read it top-down: a workspace owns members, brands, channels and telemetry; a brand owns posts; a post owns its media, approval history and publish jobs. publish_jobs is the only table with two parents - it joins a post to the channel it goes out on.




*Arrow = foreign key, pointing at the parent table.*

*Grey boxes have no parent in the ownership chain (global or FK-free by design).*

*Not shown: posts.created_by and approval_logs.user_id also point to users.*

# 3. Table by table

Each section says what the table is for, what the important columns mean, and how it connects to the rest. Column types are abbreviated; the migration file is the source of truth.

## 3.1 users

One row per login. Workspaces, posts and approvals all point back here.

| Column | Type | Meaning |
|---|---|---|
| id | UUID PK | Primary identifier |
| email | VARCHAR(255) UNIQUE | Login identifier, stored lower-case |
| password_hash | VARCHAR(255) | bcrypt (cost 12). Never the plain password |
| full_name, avatar_url | VARCHAR | Display data |
| is_active | BOOLEAN | Soft disable: an inactive user cannot log in or refresh |
| created_at, updated_at | TIMESTAMPTZ | Audit timestamps |

**Relationships:** has many refresh_tokens, workspace_memberships; referenced by workspaces.owner_id, posts.created_by, approval_logs.user_id.

## 3.2 refresh_tokens

Server-side session store for the long-lived login cookie. The raw token lives only in an HttpOnly cookie; the database keeps its SHA-256 hash so a leaked dump cannot replay sessions.

| Column | Type | Meaning |
|---|---|---|
| user_id | UUID FK -> users (CASCADE) | Token owner |
| token_hash | VARCHAR(64) UNIQUE | SHA-256 of the raw token |
| expires_at | TIMESTAMPTZ | Hard expiry (14 days by default) |
| is_revoked | BOOLEAN | Set on logout and on every rotation (each refresh issues a new row) |
| ip_address, user_agent | VARCHAR(45), TEXT | For a future 'active sessions' screen |

**Relationships:** belongs to users.

Rotation: /auth/refresh revokes the presented token and inserts a new one. Expect many revoked rows per user; a cleanup job can purge rows where expires_at < now().

## 3.3 workspaces

The tenant. A workspace is a company or team account: it owns members, brands and connected social accounts. Every user gets one workspace automatically on sign-up.

| Column | Type | Meaning |
|---|---|---|
| name | VARCHAR(255) | Team or company name |
| slug | VARCHAR(100) UNIQUE | URL-safe identifier, generated as name + timestamp |
| owner_id | UUID FK -> users | The founding user; also has an OWNER row in workspace_members |
| allow_self_approval | BOOLEAN default true | True: a post's author may approve it (one-person business). False: author != approver is enforced (agency). Owner-only setting |

**Relationships:** has many workspace_members, brands, social_accounts, usage_logs.

## 3.4 workspace_members

Who belongs to a workspace and with what role. This table is the **authorisation backbone**: every workspace-scoped request is checked against it.

| Column | Type | Meaning |
|---|---|---|
| workspace_id, user_id | UUID FKs (CASCADE) | UNIQUE together: one role per user per workspace |
| role | ENUM workspace_role | owner \| admin \| member |
| joined_at | TIMESTAMPTZ | When added |

**Relationships:** joins users to workspaces (many-to-many).

Role rules in the API: **owner** can do everything incl. grant admin; **admin** can approve/reject posts, connect accounts, add members; **member** can create brands and posts and submit for approval. An approver can never approve their own post.

## 3.5 brands

A brand is one business identity the team writes posts for - a client (for an agency) or a product line (for a single company). It carries the style profile that posts, and later the AI, use.

| Column | Type | Meaning |
|---|---|---|
| workspace_id | UUID FK -> workspaces (CASCADE) | Owning tenant |
| name, industry | VARCHAR | industry also drives caption_templates suggestions |
| target_audience | TEXT | Persona summary |
| tone_of_voice, language | VARCHAR | e.g. 'friendly', 'en' |
| logo_url | VARCHAR(500) | Brand logo |
| brand_guidelines | JSONB | Palette, fonts, banned words - free-form |
| is_active | BOOLEAN | DELETE /brands/{id} deactivates rather than deletes |

**Relationships:** belongs to workspaces; has many posts.

**Workspace vs brand:** the workspace answers *who is working* (login, roles, connected pages); the brand answers *for whom the post is written*. An agency has one workspace and many brands; a small business usually has one of each.

## 3.6 posts

The content core. One row per piece of content, carrying its approval state and, after publishing, a filtered summary of the result.

| Column | Type | Meaning |
|---|---|---|
| brand_id | UUID FK -> brands (CASCADE) | Owning brand (and through it, the workspace) |
| created_by | UUID FK -> users | Author - used for the 'cannot approve own post' rule |
| title, caption, hashtags, cta | VARCHAR / TEXT / JSONB | The content. hashtags is a JSON array of strings |
| status | ENUM post_status | draft \| pending_approval \| approved \| scheduled \| publishing \| published \| partially_published \| failed |
| content_hash | VARCHAR(64) | SHA-256 of caption + sorted media IDs. Approval is bound to this value |
| version_number | INTEGER | Increments whenever the hash changes |
| platform_targets | JSONB | Per-platform caption/hashtag overrides (reserved) |

| Column | Type | Meaning |
|---|---|---|
| scheduled_at, timezone | TIMESTAMPTZ, VARCHAR | When to publish (UTC) and the user's display zone |
| published_at, publish_result | TIMESTAMPTZ, JSONB | Set by the worker: {platform: {platform_post_id, url}} |
| retry_count | INTEGER | How many worker retries happened |

**Relationships:** belongs to brands; has many post_media, approval_logs, publish_jobs.

Two CHECK constraints guard the state machine at the database level: a *scheduled* post must have scheduled_at, and a publishing/published post must have a content_hash.

## 3.7 post_media

Images or videos attached to a post. Stores a **storage key**, not a URL - URLs are generated on demand (local path in dev, presigned S3 link in production), so an expiring link never changes the post's content hash.

| Column | Type | Meaning |
|---|---|---|
| post_id | UUID FK -> posts (CASCADE) | Parent post |
| storage_key | VARCHAR(1000) | e.g. media/3f9c...a1.png |
| media_type | ENUM media_type | image \| video \| carousel_item |
| sort_order | INTEGER | Carousel order |
| alt_text | VARCHAR(500) | Accessibility text |
| metadata | JSONB | Original filename, size, MIME |

**Relationships:** belongs to posts. Adding or removing a row changes the post's content hash and invalidates approval.

## 3.8 approval_logs

Append-only record of every submit / approve / reject / invalidate on a post, with the content hash at that moment. This is how the system proves *what* was approved and *by whom*.

| Column | Type | Meaning |
|---|---|---|
| post_id | UUID FK -> posts (CASCADE) | Target post |
| user_id | UUID FK -> users | Acting user |
| action | ENUM approval_action | submitted \| approved \| rejected \| invalidated |
| content_hash_at_action | VARCHAR(64) | Hash of the content when the action happened |
| rejection_reason | TEXT | Required on reject; also used for the invalidation message |

**Relationships:** belongs to posts.

Never UPDATE or DELETE rows here. Before publishing, the worker recomputes the post's hash and compares it to the latest *approved* row - if they differ, the job fails instead of posting un-reviewed content.

## 3.9 social_accounts

A connected channel: a Facebook page, Instagram business account or LinkedIn profile. Belongs to the workspace, not to a brand - any brand in the workspace may publish through it.

| Column | Type | Meaning |
|---|---|---|
| workspace_id | UUID FK -> workspaces (CASCADE) | Owning tenant |

| Column | Type | Meaning |
|---|---|---|
| platform | ENUM social_platform | facebook \| instagram \| linkedin |
| platform_account_id | VARCHAR(255) | The page / profile ID on the platform. UNIQUE with workspace + platform |
| account_name, account_type | VARCHAR | Display name; page / professional / company |
| access_token_encrypted | TEXT | OAuth token, Fernet-encrypted at rest (AES-128-CBC + HMAC) |
| refresh_token_encrypted | TEXT | Where the platform supports refresh (LinkedIn, Phase 2) |
| token_expires_at | TIMESTAMPTZ | Checked every 6 h by a Celery task |
| platform_metadata | JSONB | page_id, ig_user_id, author_urn - what the adapter needs |
| is_active | BOOLEAN | Disconnect = set false (row kept so old jobs still resolve) |
| needs_reauth | BOOLEAN | Set when the token is about to expire; the UI shows a reconnect banner |

**Relationships:** belongs to workspaces; has many publish_jobs.

Tokens are never returned by the API. Decryption happens only inside the publish worker.

## 3.10 publish_jobs

One row per (post, social account) to publish. Jobs own their own status; the post's status is derived from all of its jobs. A Celery Beat task scans this table every minute and dispatches due rows.

| Column | Type | Meaning |
|---|---|---|
| post_id | UUID FK -> posts (CASCADE) | What to publish |
| social_account_id | UUID FK -> social_accounts | Where to publish |
| celery_task_id | VARCHAR(255) | NULL until dispatched - the scan only picks NULL rows, so nothing runs twice |
| status | ENUM job_status | queued \| processing \| completed \| failed \| cancelled \| needs_review |
| result | JSONB | Filtered platform response: platform_post_id, url, published_at |
| error_message | TEXT | Why it failed or was cancelled |
| attempt_number | INTEGER | 1 on first run; retries increment |
| scheduled_for | TIMESTAMPTZ | When it becomes due |
| started_at, processed_at | TIMESTAMPTZ | started_at is set before the platform call; a job stuck in processing for 1 h is flagged needs_review |

**Relationships:** belongs to posts and social_accounts.

**needs_review** means 'a worker crashed around the platform call and we do not know if it posted'. A human checks the platform before retrying - this avoids double-posting to a customer's page.

## 3.11 usage_logs

Lightweight telemetry per workspace: posts created, media uploaded, posts published. Intended for quotas and future billing.

| Column | Type | Meaning |
|---|---|---|
| workspace_id | UUID FK -> workspaces (CASCADE) | Tenant |
| user_id | UUID FK -> users, nullable | NULL for system/worker events |
| operation | ENUM operation_type | post_created \| post_published \| media_uploaded \| ai_generation |
| metadata | JSONB | post_id, byte counts, etc. |

**Relationships:** belongs to workspaces.

## 3.12 caption_templates

Global, read-only library of 'Smart Templates' (the MVP stand-in for AI writing). Shared by every tenant, so there is no workspace_id and no write endpoint - content comes from `seeds/seed.py`.

| Column | Type | Meaning |
|---|---|---|
| industry | VARCHAR(100) | Matched against brands.industry |
| category | ENUM template_category | promotional \| educational \| engagement \| seasonal |
| title, template_text | VARCHAR, TEXT | Body with {{placeholder}} tags |
| suggested_hashtags, suggested_cta | JSONB, VARCHAR | Presets to copy into a post |
| is_active | BOOLEAN | Hide without deleting |

**Relationships:** no foreign keys - a standalone lookup table.

## 3.13 audit_events

Universal, tamper-evident ledger of who did what, across every entity. It has **zero foreign keys on purpose**: FK-linked rows are cascade-deleted with their parent, and this table must survive deleting a workspace, user or post.

| Column | Type | Meaning |
|---|---|---|
| workspace_id, user_id | UUID (no FK) | Plain values - snapshots |
| user_email, user_name | VARCHAR(255) | Actor snapshot at action time (survives user rename/deletion) |
| entity_type, entity_id | VARCHAR(50), UUID | Post \| Brand \| Workspace \| SocialAccount \| WorkspaceMember |
| action | ENUM audit_action | CREATE UPDATE DELETE PUBLISH SUBMIT APPROVE REJECT SCHEDULE CONNECT DISCONNECT INVITE ROLE_CHANGE |
| changes | JSONB | {"before": {...}, "after": {...}} |
| ip_address, user_agent | VARCHAR(45), TEXT | Forensics |

**Relationships:** none - deliberately. Write via app/services/audit.py::record_audit_event, which never raises.

approval_logs vs audit_events: approval_logs is post-specific and dies with the post; audit_events covers every entity and is permanent. Both are append-only.

# 4. All relationships in one list

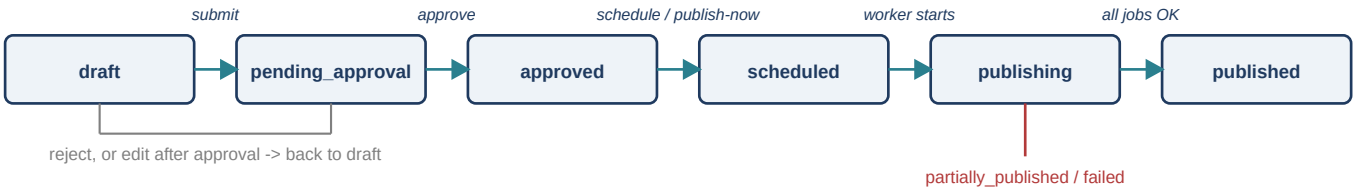
| Child.column | Parent | On parent delete | Meaning |
|---|---|---|---|
| refresh_tokens.user_id | users.id | CASCADE | A user's sessions |
| workspaces.owner_id | users.id | RESTRICT | Founding user (cannot delete a user who owns a workspace) |
| workspace_members.workspace_id | workspaces.id | CASCADE | Membership |
| workspace_members.user_id | users.id | CASCADE | Membership |
| brands.workspace_id | workspaces.id | CASCADE | Brand belongs to a tenant |
| social_accounts.workspace_id | workspaces.id | CASCADE | Channel belongs to a tenant |
| posts.brand_id | brands.id | CASCADE | Post belongs to a brand |
| posts.created_by | users.id | RESTRICT | Author |
| post_media.post_id | posts.id | CASCADE | Attachment |
| approval_logs.post_id | posts.id | CASCADE | Approval history |
| approval_logs.user_id | users.id | RESTRICT | Approver / submitter |
| publish_jobs.post_id | posts.id | CASCADE | Dispatch unit |
| publish_jobs.social_account_id | social_accounts.id | RESTRICT | Target channel |
| usage_logs.workspace_id | workspaces.id | CASCADE | Telemetry |
| usage_logs.user_id | users.id | RESTRICT | Actor (nullable) |
| audit_events.* | - | - | No FKs by design |
| caption_templates | - | - | No FKs; global |

RESTRICT here means the database default: the delete fails while children exist. In practice users are deactivated (is_active = false), never deleted.

### *Unique constraints*

| Table | Columns | Why |
|---|---|---|
| users | email | One account per email |
| workspaces | slug | URL identifier |
| refresh_tokens | token_hash | Token lookup |
| workspace_members | workspace_id + user_id | One role per person per workspace |
| social_accounts | workspace_id + platform + platform_account_id | Same page connected twice would publish every post twice |

# 5. Post lifecycle and which tables change




| Step | API | Rows written |
|---|---|---|
| Create | POST /brands/{id}/posts | posts (draft), usage_logs, audit_events |
| Upload media | POST /media/upload | post_media; posts.content_hash + version_number |
| Submit | POST /posts/{id}/submit | posts.status=pending_approval, approval_logs(submitted), audit_events |
| Approve / reject | POST /posts/{id}/approve \| reject | posts.status, approval_logs(approved \| rejected), audit_events. Approver must be admin/owner and not the author |
| Schedule / publish now | POST /posts/{id}/schedule \| publish-now | posts.status=scheduled + scheduled_at; one publish_jobs row per target account; audit_events |
| Edit after approval | PATCH /posts/{id} | posts back to draft, approval_logs(invalidated), open publish_jobs -> cancelled |
| Worker runs | Celery (every minute) | publish_jobs.status processing -> completed/failed; posts.status + publish_result; usage_logs; audit_events |

# 6. Conventions and gotchas

| Topic | Rule |
|---|---|
| Primary keys | UUID v4 generated in Python (uuid.uuid4). Never sequential integers. |
| Enums | PostgreSQL enum types storing the lower-case VALUE ('draft'), not the Python name ('DRAFT'). Add a value = new Alembic migration with ALTER TYPE ... ADD VALUE. |
| Timestamps | Always TIMESTAMPTZ, always UTC. posts.timezone is display-only. |
| JSONB defaults | Server default '{}' or '[]'; never NULL for hashtags, brand_guidelines, metadata. |
| Soft deletes | brands, social_accounts and users use is_active. posts are hard-deleted (cascade). |
| Append-only | approval_logs and audit_events: INSERT only. No code path updates or deletes them. |
| Schema changes | Edit the model in app/models/, then `alembic revision --autogenerate`, review, `alembic upgrade head`. create_all runs only in ENVIRONMENT=development as a convenience. |
| Two sessions | FastAPI uses the async engine (asyncpg); Celery workers use the sync engine (psycopg2). Write queries in 2.0 style - select(Model).where(...) - so they run on both. |
| Secrets | Never stored in plain text: passwords are bcrypt hashes, refresh tokens are SHA-256 hashes, OAuth tokens are Fernet-encrypted with FERNET_KEY. |

### *Files to read next*

| Path | What you will find |
|---|---|
| backend/app/models/*.py | SQLAlchemy models - the authoritative column list |
| backend/alembic/versions/ | 0001 creates everything; 0002 adds workspaces.allow_self_approval |
| backend/app/core/dependencies.py | Tenancy checks every route goes through |
| backend/app/api/v1/posts.py | The approval and scheduling state machine |
| backend/app/workers/tasks/publish.py | How publish_jobs are dispatched and finalized |
| backend/seeds/seed.py | Caption templates seed data |